

The generalized linear model

Jure Demšar and Erik Štrumbelj, University of Ljubljana

/ Summary

- The intercept (α or β_0) and slope parameters (β_i) in simple linear regression are easy to interpret and can as such provide key insight into our data generating process.
- When modeling the relationship between independent and dependent variables, simple linear regression often gives suboptimal results. We need a more powerful tool, especially so in cases where the dependent variable is not metric.
- Generalized linear model (GLM) is a powerful tool capable of working with dependent variables of various scale types (metric, ordinal, nominal and count) and error distributions other than normal.
- The simple linear model is actually the simplest of all GLMs.
- In the GLM, beta coefficients are usually not as easy to interpret as in the case of simple linear regression. Fortunately, we only need some relatively easy math to make them understandable!

// Simple linear regression

Simple linear regression traditionally denotes a linear regression model that has a single independent variable. In the following example we will use Bayesian linear regression to model temperature data through ages (the data is stored in `temperature.csv`). The dataset contains three columns, `temperature` denotes the average temperature, while `year` and `month` depict the time of the measurement. As we are interested in the yearly average temperature through ages, we will average the temperature over months. As a result, our dependent variable (`y`) will be the average yearly temperature and our independent variable (`x`) will be the year. We will use a linear normal model (or linear normal regression) to model the dependency between `x` and `y`. At the same time we will see how one would build a model for normally distributed data. This is the Stan model we will use:

```
data {
  int<lower=1> n; // total number of data points
  vector[n] x;   // x values
  vector[n] y;   // y values
}

parameters {
  real a;        // intercept
  real b;        // slope
  real<lower=0> sigma; // stdev
}

model {
  // model
  y ~ normal(a + b * x, sigma);
}
```

The `data` block contains three variables, `n` stores the total number of data points, `x` the values of the independent variable and `y` the values of the `dependent` variables. In the `model` block you can

see that we are using a normal distribution as the core building block. To model a linear dependency between `x` and `y` we replaced the `mean` (μ) parameter of the normal distribution with a linear function (`a + b * x`), where `a` denotes the intercept and `b` the slope of the function. The `parameters` block contains the three parameters of the model that we are interested in estimating. Below is the code that compiles the model, loads and prepares the data and fits the model.

```
# load libraries
library(cmdstanr)
library(tidyverse)
library(bayesplot)
library(posterior)
library(mcmcse)

# compile the model
model <- cmdstan_model("./models/simple_linear.stan")

# prepare the data
data <- read.csv("./data/temperature.csv", sep = ";")

# remove month
data <- data %>% select(-month)

# mean per year
data <- data %>%
  group_by(year) %>%
  summarise(temperature = mean(temperature))

# x shift for numerical stability
min_year <- min(data$year)
x <- data$year - min_year
y <- data$temperature

# prepare input for Stan
stan_data <- list(n = nrow(data), x = x, y = y)

# fit
fit <- model$sample(
  data = stan_data,
  seed = 1
)

# traceplot
mcmc_trace(fit$draws())

# summary
fit$summary()
```

The `traceplot` and `summary` both show no reasons for any concerns, we will skip them here for brevity, but in practice you should always start your analysis by inspecting the outputs of these two functions. Whenever possible you should also always visually inspect how fitted distributions match the underlying data. With more complex and highly dimensional models this most likely will not be possible but here we can easily do this. Let us take a peek at the value of our model's parameters (the intercept, `a` and the slope `b`).

```
# params
df <- as_draws_df(fit$draws())
mcse(df$a)
```

```
## $est
## [1] 8.011116
##
## $se
## [1] 0.002670877
```

```
mcse(df$b)
```

```
## $est
## [1] 0.01322325
##
## $se
## [1] 3.918446e-05
```

```
mcse(df$b > 0)
```

```
## [1] "No variability observed in a component. Setting batch size to 1"
```

```
## $est
## [1] 1
##
## $se
## [1] 0
```

The intercept (α or **a** in our case) denotes the value of **y** when **x** equals zero. For the purpose of numerical stability and to make life easier for Stan, we have shifted our data, this now means that **a** denotes the temperature at the minimum year in our dataset (1901). The slope parameter (β or **b** in our case) denotes how **y** changes with **x**, in our case how temperature changes through ages. You can see that we are extremely confident ($P \approx 1$) that value of **b** is positive. This means that the temperature has been rising over the last 120 years, in fact it has been rising on average by 0.013 ± 0.0004 °C per year.

```
# plot only 100 random regression lines
df_100 <- sample_n(df, 100)

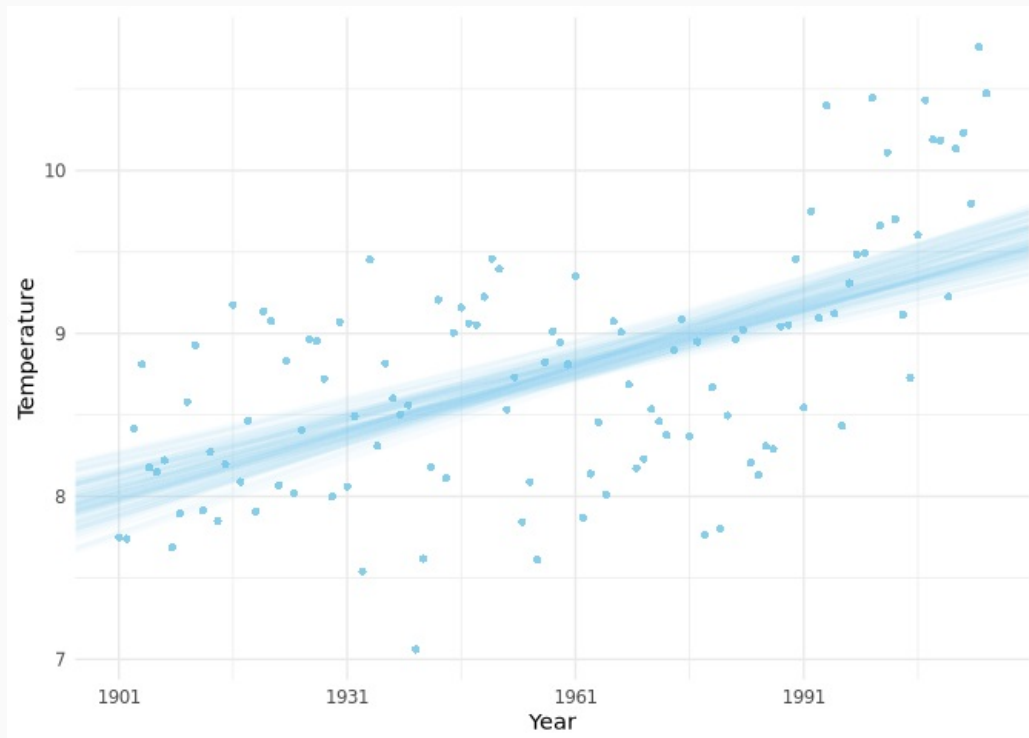
# x axis
x_breaks <- seq(from = 0, to = 120, length.out = 5)
x_labels <- x_breaks + min_year

# visualize data points with regression lines in the background
ggplot() +
  geom_point(
    data = data,
```

```

aes(x = x, y = y),
shape = 16,
color = "skyblue"
) +
geom_abline(
  data = df_100,
  aes(slope = b, intercept = a),
  alpha = 0.05,
  linewidth = 1,
  color = "skyblue"
) +
theme_minimal() +
labs(x = "Year", y = "Temperature") +
scale_x_continuous(breaks = x_breaks, labels = x_labels)

```



// Multiple linear regression

Multiple linear regression is a logical next step after getting to know the simple linear regression model. The only difference between the two is that multiple linear regression can handle input data that has more than one independent variable. As a result, we can check the correlation between the dependant variable (y) and many independent variables ($x_1, x_2, \dots, x_k$) simultaneously. The upgrade is quite easy. First, we have a new entry in the input data block, where m denotes the number of independent variables. Second, our input variables are now stored in a matrix (X), since there are many of them. And third, we have multiple β coefficients, one for each independent variable. Below is an example of multiple linear regression as a .stan model:

```

data {
  int<lower=1> n; // total number of data points
  int<lower=1> m; // number of predictors
  matrix[n, m] X; // independent variables
  vector[n] y; // dependent variable
}

parameters {
  real a; // intercept

```



```

vector[m] b;          // slope
real<lower=0> sigma; // stdev
}

model {
  // model
  y ~ normal(a + X * b, sigma);
}

```

// Parameters of linear regression models

Typically linear regression models have several parameters, the number of parameters depends on the number of independent variables and their interactions. For the purpose of this explanation let us use the 50 startups datasets that includes information about the profits of startup companies in the USA. In this example we will investigate the relationship between resources invested in research, administration and marketing and the company's profit. For simplicity sake, we will discard the location variable.

The gist of our problem is to figure out how we should distribute our resources if we just opened a startup company, in other words we are interested in the ratio between funds spent for research, administration and marketing that is expected to generate the most profit. We will use Bayesian multiple linear regression (we use the term simple linear regression when there is only one independent variable and the term multiple linear regression if we have more than one independent variable):

$$y_i | x_i, \alpha, \beta_r, \beta_a, \beta_m, \sigma \sim N(\alpha + \beta_r x_{r,i} + \beta_a x_{a,i} + \beta_m x_{m,i}, \sigma),$$

$$\beta_r, \beta_a, \beta_m \sim N(0, 1).$$

// The intercept

α (or often β_0) represents the intercept, the value of the dependent variable (y) when all independent variables (x) are set to zero. There are cases where removing the α parameter might make sense, with this we would make the model less flexible by fixing the value of the dependent variable to 0 when all independent variables are 0. There is a strong argument to do this in our case since without any investments there should probably be no profit:

$$y_i | x_i, \beta_r, \beta_a, \beta_m, \sigma \sim N(\beta_r x_{r,i} + \beta_a x_{a,i} + \beta_m x_{m,i}, \sigma).$$

Note though, that the intercept often does make a lot of sense. For example if we were interested in explaining relationship between salaries and work experience (e.g., years of employment) then the intercept would denote the average salary for a person without any work experience (0 years of work experience).

// The β coefficients

The β coefficients are the core of regression models as they describe the relationship between independent and dependent variables. The table below shows some summary data for β coefficients in our case.

Variable	Mean	$q_{0.5}$	$q_{0.95}$
β_r	0.714 ± 0.002	0.603	0.824
β_a	0.329 ± 0.001	0.277	0.382

β_m

0.083 ± 0.001

0.045

0.121

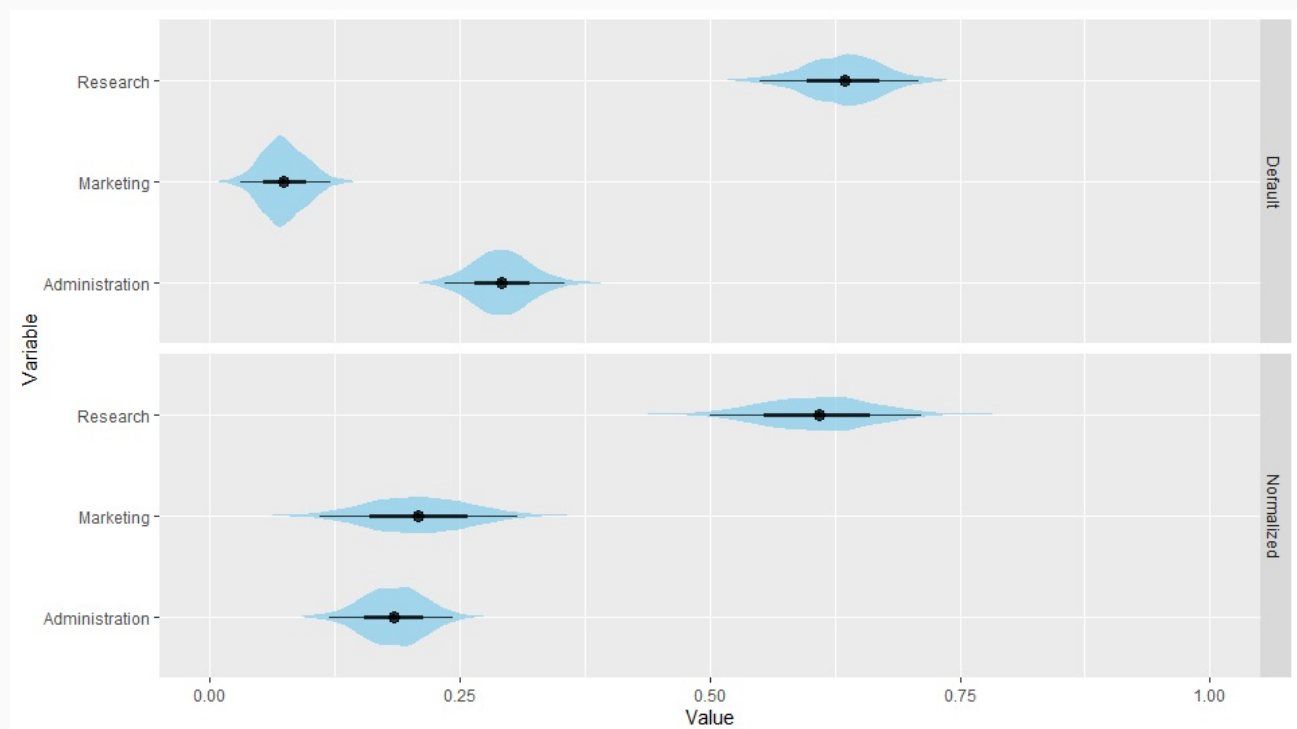
A positive value of a β coefficient denotes a positive correlation between the independent while the opposite holds for a negative of a β coefficient. For example, the value of β_r tells us that investing in research has a positive influence on profit. Furthermore, the value of 0.714 denotes that \$1 invested in research will generate a profit of \$0.714 on average. As we can see, investing in all three aspects of a business generates profit, this makes sense as if something is not valuable for a company they most likely would not do it. When independent variables are on the same scale we can compare their effects through β coefficients.

In our case all three independent variables use the same scale – money invested in USD (\$) – meaning that we can directly compare these coefficients. For example, β_r is approximately 2.2 times larger than β_a , this means that resources invested in research should generate approximately 2.2 times as much profit as resources invested in administration. However, if we used another independent variable, for example happiness of employees measured on a scale from 1 to 10, its β coefficient could not be directly comparable with the other three as it would reside on a completely different scale.

When preparing data for (G)LMs we often standardize or normalize it, when doing so we have to pay attention to the purpose of our regression. Let us illustrate this on our case by comparing results between two cases, in the first case we will use input data as is, and in the second data we will normalize each column independently:

$$x_i = \frac{x_i - \min(x)}{\max(x) - \min(x)}.$$

To calculate the optimal investment ratio we calculate the ratio between samples for β coefficients. The visualization below compares this ratio between the analysis where we left the input data as is (marked as Default) and the analysis where we normalized the input data.



If we compare findings between the two methods we can spot significant differences:

	Default	Normalized
Research	0.632 ± 0.001	0.608 ± 0.001

Marketing	0.293 ± 0.001	0.184 ± 0.001
Administration	0.075 ± 0.001	0.208 ± 0.001

What happened here? When normalizing the data we changed the ratio between research, administration and marketing investments in the input data. The reason for that lies in the fact that maximum and minimum values in columns are different! Since in the normalized case the input data does not reflect the proper ratio between spending of startup companies this input data cannot be used to get the answer we are interested in.

On the other hand, if we are interested in developing a model that is capable of estimating profits of new companies given their spending then normalization or standardization of input variables is a perfectly viable path to take. In this case we no longer care what the ratio between β coefficients is, instead we are probably interested only in getting an estimator that is as accurate as possible. The point of this example was to emphasize that we have to be careful when performing various transformation on input data since these transformations might change the results of our analysis. Like previously mentioned, we will often regularize or standardize our input data, but as illustrated in some cases we have to be extra careful when doing this. Before we move onto GLMs let us briefly touch three additional subjects.

// Polynomial regression

As you can image, we have a lot of freedom when designing our regression models. We could model our data with this model:

$$y_i \mid x_i, \beta_3, \beta_2, \beta_1, \sigma \sim N(\beta_3 x_{r,i}^3 + \beta_2 x_{r,i}^2 + \beta_1 x_{r,i}, \sigma).$$

This is called polynomial regression and still falls in the category of linear models, we achieved non-linearity by transforming independent variables (something we will do a lot). This could be a completely valid model and might even fit our input data better than the one we used previously. However, making sense of this model's outputs would be very hard since the β coefficients are now much harder to interpret. It is important to always have in mind that in Bayesian analysis we are (usually) trying to devise models that provide us with useful insights about the data generating processes in question! While complex models might fit the data extremely well they are often too hard to understand and do not provide any new insights into the data generating process. In fact, in several sensible scientific fields (such as clinical medicine and trials...) regulative agencies enforce the use of interpretable models only.

// Non-additive interactions between independent variables

When designing models it is also important to consider possible interactions between independent variables. So far, we used only additive interactions but we are often faced with data where more complex interactions are useful too. Consider, for example, a study where we are trying to model people's happiness by using their health and annual income. If someone has no income then they are most likely unhappy, regardless of their health status. Vice versa, if they are of extremely poor health, wealth most likely will not cheer them up. In such examples, consider adding a multiplicative interaction between independent variables:

$$y_i \mid x_i, \alpha, \beta_h, \beta_w, \beta_{hw}, \sigma \sim N(\alpha + \beta_h x_{h,i} + \beta_w x_{w,i} + \beta_{hw} x_{h,i} x_{w,i}, \sigma).$$

// Heavily correlated independent variables

We will use the multiple linear regression model (`models/linear.stan`) and a toy dataset (`data/toy.csv`) to illustrate the behavior of linear regression when independent variables are

heavily correlated. The `toy.csv` dataset has a single independent variable x . For this demonstration we will artificially create another one that is heavily correlated with x :

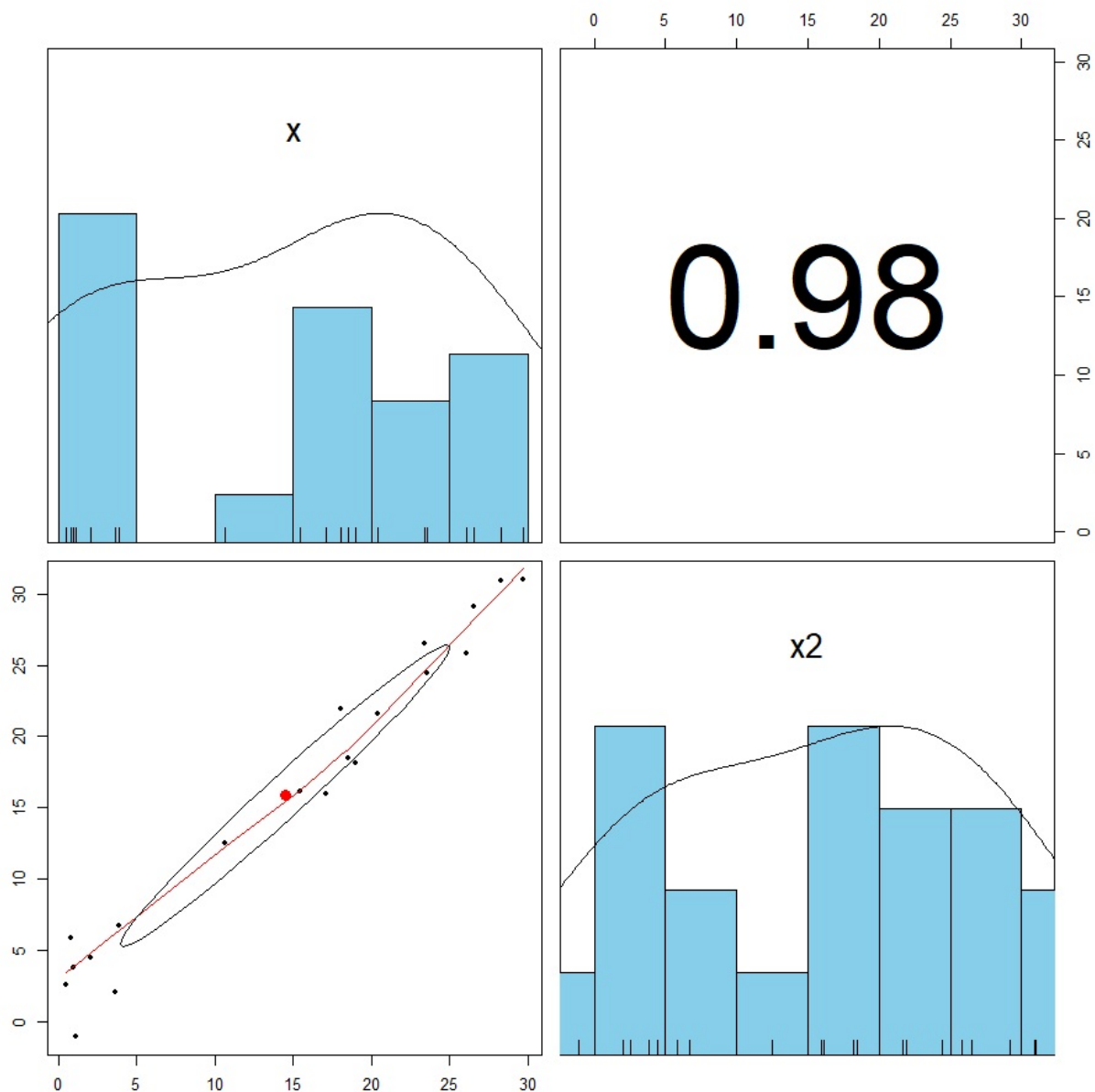
```
# load the data
data <- read.csv("./data/toy.csv")

# create dummy x2
data$x2 <- data$x + rnorm(nrow(data), 0, 2)

# independent variables only
X <- data %>% select(-y)

# correlation
# plot correlation of independent variables
pairs.panels(X,
             method = "pearson", # correlation method
             hist.col = "skyblue",
             density = TRUE, # show density plots
             ellipses = TRUE # show correlation ellipses
)
```

The above code snippet create the dummy independent variable and plots the correlation between x and x_2 . We can see that they are heavily correlated (anything else would be most surprising at this point).



Now, let us fit the model a couple of times:

Run	β_1	β_2
1	1.80 ± 0.49	-0.221 ± 0.45
2	0.892 ± 0.53	0.632 ± 0.48
3	1.31 ± 0.44	0.279 ± 0.45

You can see that the means β coefficients are quite unstable and have a large standard deviation. This is completely expected since there is an infinite number of ways that we can combine the x and x_2 independent variables to get a good fit. The purpose of this illustration is to show that while our model provides a good fit of the underlying data, the interpretation of β coefficients should be taken with a pinch of salt. Making meaningful conclusions based on our results is impossible here! How do we overcome this? We usually remove one of the correlated variables, the information it stores is extremely low since the same information is also incorporated in the other variable.

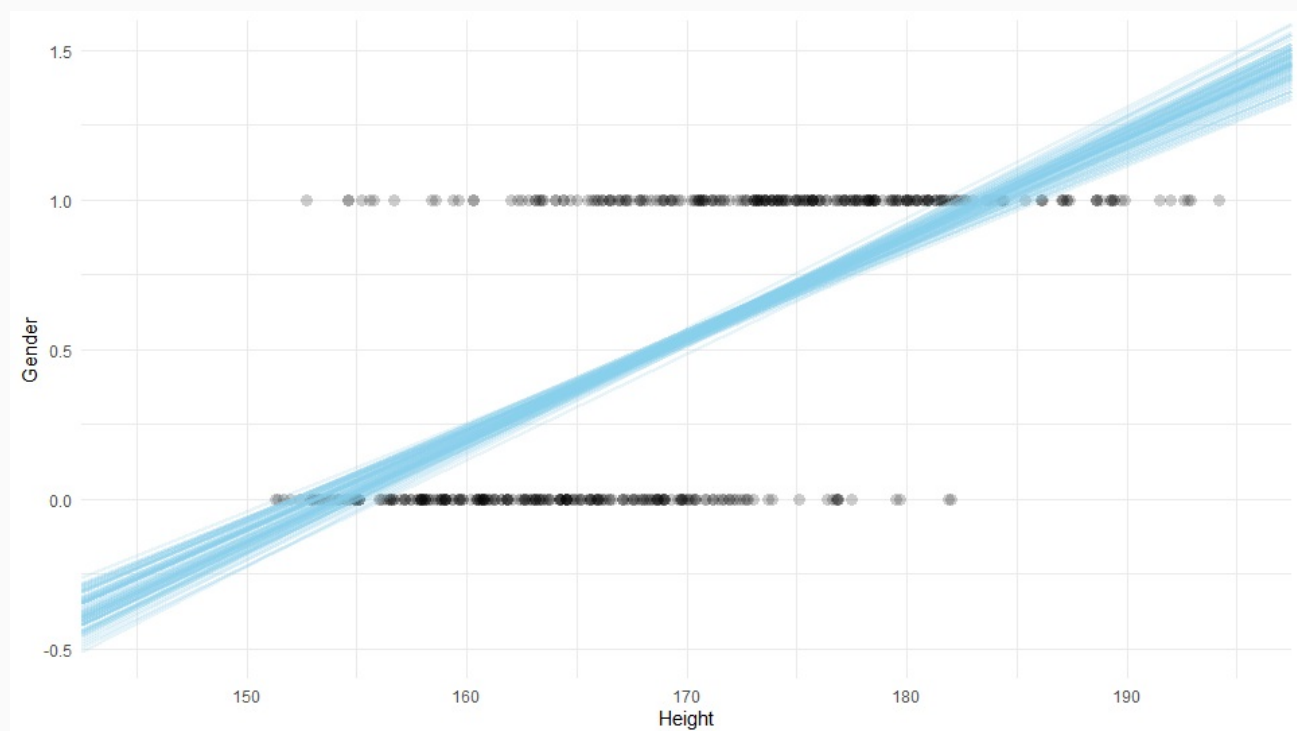
If we observe the values for each run more carefully we can see that the sum of both β coefficients is around 1.5. If you remember from the previous session, the β we used to generate this data equalled 1.5 and that would be the result if we removed one of the two problematic independent

variables.

/ The generalized linear model

The generalized linear model (GLM) is an extremely powerful and flexible tool for modeling the relationship between independent variables (also called predictors or explanatory variables) and dependent variables (also called predicted variables or response variables). The GLM generalizes linear regression by allowing the linear model to be related to the response variable via a link function and by allowing the magnitude of the variance of each measurement to be a function of its predicted value.

So far, we already got familiar with the simple linear regression, there we used the value of one metric variable to increase our understanding of another metric variable under the assumption that residuals (errors) are normally distributed. Now, let us try and use the same model for cracking a different nut. Say that we want to use height (in centimeters) of humans to predict their sex. See [models/simple_linear.stan](#) for Stan code of the model and [code/linear_binary.R](#) for R code behind this experiment. Since linear regression works with numbers, we had to label the female sex as 0 and the male sex as 1. The visualization below depicts 100 regression lines based on the posterior distributions of intercept and slope in our Bayesian analysis.



We can immediately spot a couple of issues here, the most prominent one being that now all of a sudden sex can have a value that is higher than 1 or lower than 0. We can interpret values between 0 and 1 as probabilities of being male, but what are we supposed to do with values below 0 and above 1, these outliers have no reasonable meaning! Based on the above figure we can also quite confidently claim that our fit is not the best, meaning that our model does not describe the data well. What we need here is a model suitable for modeling values that range from 0 to 1, possibly with something else than a straight line. GLMs are just that! As you will see in sections that follow, they can be used with dependent variables on all scale types (metric, ordinal, nominal and count).

// A formal expression of the GLM

We can formally define the GLM as:

$$y \mid \mu, \theta \sim p(y \mid \mu, \theta),$$

$$E[y] = \mu = f^{-1}(l(X, \alpha, \beta)),$$

$$f(\mu) = l(X, \alpha, \beta),$$

where X are independent variables combined in the linear function $l(\cdot)$ (for example $l(X, \alpha, \beta) = \alpha + X\beta$), $f(\cdot)$ is the link function and $f^{-1}(\cdot)$ its inverse. The data y are distributed around the central tendency μ and possible other parameters θ in accordance to some probability density function $p(\cdot)$.

The link function ($f(\cdot)$) and its inverse ($f^{-1}(\cdot)$) provide the relationship between the mean of the distribution function and the linear combination of independent variables. The link function should be smooth and injective. It is nice if it is also bijective, but this is not mandatory. The fact that many functions fit these limitations makes the GLM a very flexible tool. However, in practice some functions are more convenient than others. The table below lists some commonly used link functions and their inverses.

Link function	$g(\mu)$	$g^{-1}(\eta)$	
identity	μ	η	$\mathbb{R} \rightarrow \mathbb{R}$
logarithm	$\log(\mu)$	e^η	$\mathbb{R} \rightarrow (0, \infty)$
inverse	μ^{-1}	η^{-1}	$\mathbb{R} \rightarrow \mathbb{R} \setminus \{0\}$
root	$\sqrt{\mu}$	η^2	$\mathbb{R} \rightarrow [0, \infty)$
logit	$\log \frac{\mu}{1-\mu}$	$\frac{1}{1+e^{-\eta}}$	$\mathbb{R} \rightarrow (0, 1)$
probit	$\Phi^{-1}(\mu)$	$\Phi(\eta)$	$\mathbb{R} \rightarrow (0, 1)$
log-log	$-\log(-\log(\mu))$	$\exp(-\exp(-\eta))$	$\mathbb{R} \rightarrow (0, 1)$
complementary log-log	$\log(-\log(1-\mu))$	$1 - \exp(-\exp(\eta))$	$\mathbb{R} \rightarrow (0, 1)$

// Logistic regression

Let us start our venture into GLMs by building on our previous attempt of modeling relationship between person's height and sex. In our case sex is a binary variable so it would make sense to use a suitable pdf. Bernoulli's distribution was a good fit when we modeled coin tosses so let us try building a linear model with it:

$$y \mid \theta \sim \text{Bernoulli}(\theta),$$

$$f(\theta) = \alpha + x\beta,$$

$$\theta = f^{-1}(\alpha + x\beta),$$

$$y \mid x, \alpha, \beta \sim \text{Bernoulli}(f^{-1}(\alpha + x\beta)).$$

So, the last missing piece of our puzzle is an appropriate link function ($f^{-1}(\cdot)$). Our independent variable (x) resides on the real scale, while the θ parameter of Bernoulli's distribution has to be on a $[0, 1]$ interval. If we take a look at the common link functions table above we can see several suitable candidates. By far the most popular one is the logit function which gives the name to the method we are constructing – the logistic regression. Thus our model becomes:

$$\text{logit}(\theta) = \alpha + x\beta,$$

$$\theta = \text{inv_logit}(\alpha + x\beta),$$

$$y \mid x, \alpha, \beta \sim \text{Bernoulli}(\text{inv_logit}(\alpha + x\beta)),$$

$$y \sim \text{Bernoulli}\left(\frac{1}{1 + e^{-(\alpha + x\beta)}}\right).$$

Below is model's code in Stan, as you can see the Bernoulli logit regression is so common that Stan already has it implemented. Using already implemented distributions in Stan is heavily advisable as

they are optimized in many ways.

```
data {  
  int<lower=1> n;                // number of observations  
  vector<lower=0>[n] x;          // height  
  array[n] int<lower=0, upper=1> y; // sex  
}  
  
parameters {  
  real beta; // beta  
  real alpha; // alpha  
}  
  
model {  
  // bernoulli logit  
  y ~ bernoulli_logit(alpha + beta * x);  
}
```

Alternatively `y ~ bernoulli_logit(alpha + beta * x);` could be replaced with

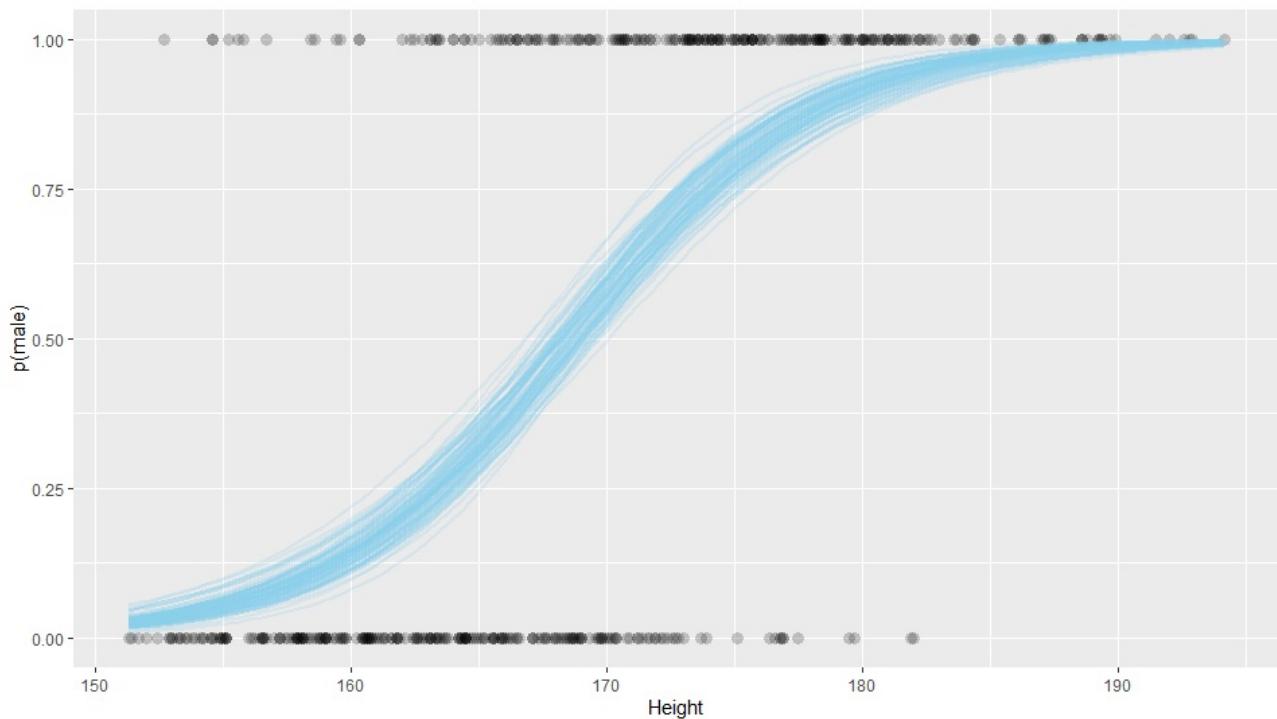
```
y ~ bernoulli(inv_logit(alpha + beta * x));
```

or even with

```
for (i in 1:n)  
  y[i] ~ bernoulli(1 / (1 + exp(-(alpha + x[i] * beta))));
```

All of the solutions above should give practically equivalent results. However, the alternative ones are slower and less stable.

Since data preparation and basic diagnostics are similar as in our previous examples we will omit them here for the sake of brevity. But do note that we executed all the steps and everything looked OK! You should never ever skip the diagnostics when doing Bayesian analysis in practice! Like already mentioned, if possible it is always good to visualize how the posterior distribution fits the underlying data:



We can see that this model is much better than the previous one – it fits the underlying data better and we no longer get probabilities below 0 or above 1. Next, let us take a look at the posterior parameters:

Variable	Mean	$q_{0.5}$	$q_{0.95}$
α	-34.9 ± 0.132	-40.6	-30.4
β	0.21 ± 0.001	0.180	0.241

Does the above β value mean that with a 1 cm increase in height the probability of being male increases for 0.21?

$$\Pr(\text{male} | \text{height} = 183 \text{ cm}) = \Pr_{183} = \frac{1}{1 + e^{-(-34.9 + 183 \cdot 0.21)}} = 0.972$$

$$\Pr(\text{male} | \text{height} = 184 \text{ cm}) = \Pr_{184} = \frac{1}{1 + e^{-(-34.9 + 184 \cdot 0.21)}} = 0.978$$

The difference between the two probabilities is 0.006, which is not equal to our β . But what does β tell us then? Things should become clear if we write down the formula for θ :

$$\theta = \text{inv_logit}(\alpha + x\beta),$$

$$\text{logit}(\theta) = \alpha + x\beta.$$

The formula implies that if height (x) increases by 1 cm, then $\text{logit}(\theta)$ goes up by β . So, to understand β coefficients we need to understand the logit function. In our case θ represents the probability that the person is male ($y = 1$), therefore we can write:

$$\text{logit}(\theta) = \text{logit}(\Pr(y = 1)) = \log\left(\frac{\Pr(y = 1)}{1 - \Pr(y = 1)}\right) = \log\left(\frac{\Pr(y = 1)}{\Pr(y = 0)}\right).$$

Since the ratio $\frac{\Pr(y=1)}{\Pr(y=0)}$ represents the odds of outcome 1 to outcome 0, then $\text{logit}(\theta)$ represents the log odds of outcome 1 to outcome 0. In other words, β denotes the change in log odds if x changes by 1! Note here, that a constant increase in log odds does not denote a constant increase in probability.

Indeed, if we take a look at the difference between log odds of Pr_{183} and Pr_{184} :

$$\log\left(\frac{\text{Pr}_{184}}{1 - \text{Pr}_{184}}\right) - \log\left(\frac{\text{Pr}_{183}}{1 - \text{Pr}_{183}}\right) = 0.21$$

we get the value of β .

// Poisson regression

Our next GLM will be the Poisson regression which assumes that the dependent variable is composed of Poisson distributed integers. We will use the logarithm as the link function which means that the inverse link function is the exponent:

$$y \mid x, \alpha, \beta \sim \text{Poisson}(e^{\alpha + X\beta}),$$

Below is Stan code for the Poisson regression, note that we use the `generated quantities` block to calculate the values of λ and predictions on the fly:

```
data {
  int<lower=1> n;          // number of data points
  int<lower=0> m;          // number of independent variables
  matrix[n,m] X;          // independent variables
  array[n] int<lower=0> y; // dependent variable
}

parameters {
  real alpha;
  vector[m] beta;
}

model {
  // poisson model
  for (i in 1:n) {
    y[i] ~ poisson(exp(alpha + X[i] * beta));
  }
}

generated quantities {
  // lambdas
  vector<lower=0>[n] lambda;

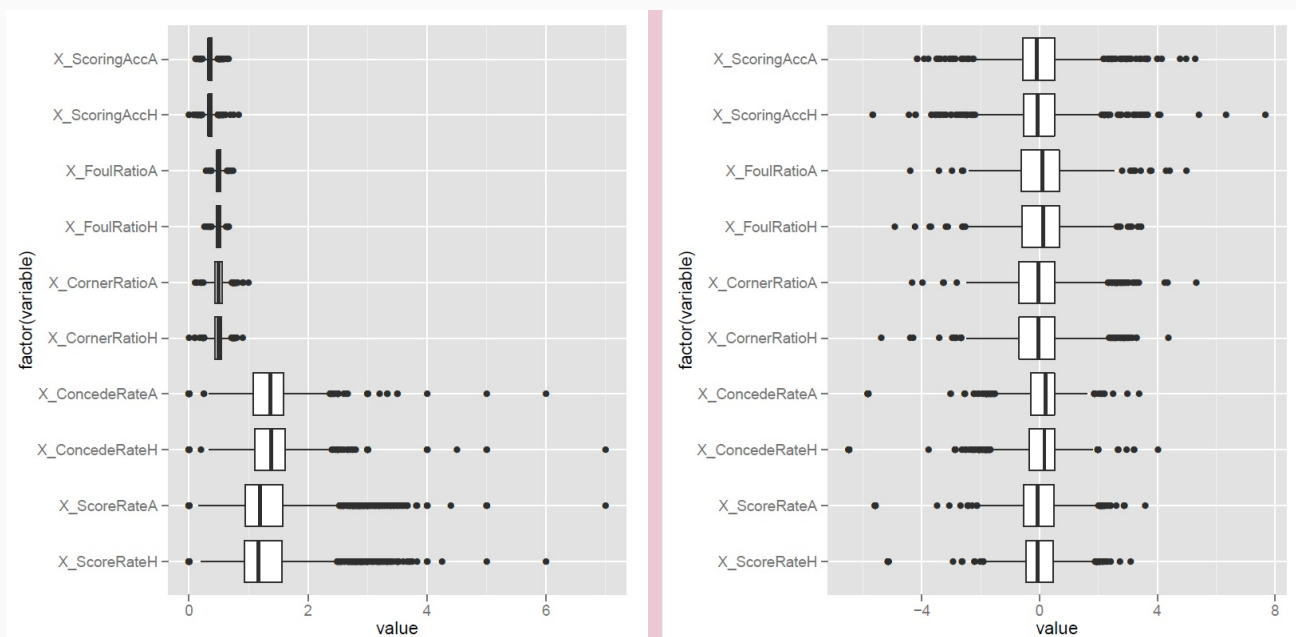
  // predictions
  array[n] int<lower=0> pred;

  for (i in 1:n) {
    lambda[i] = exp(alpha + X[i] * beta);
    pred[i] = poisson_rng(lambda[i]);
  }
}
```

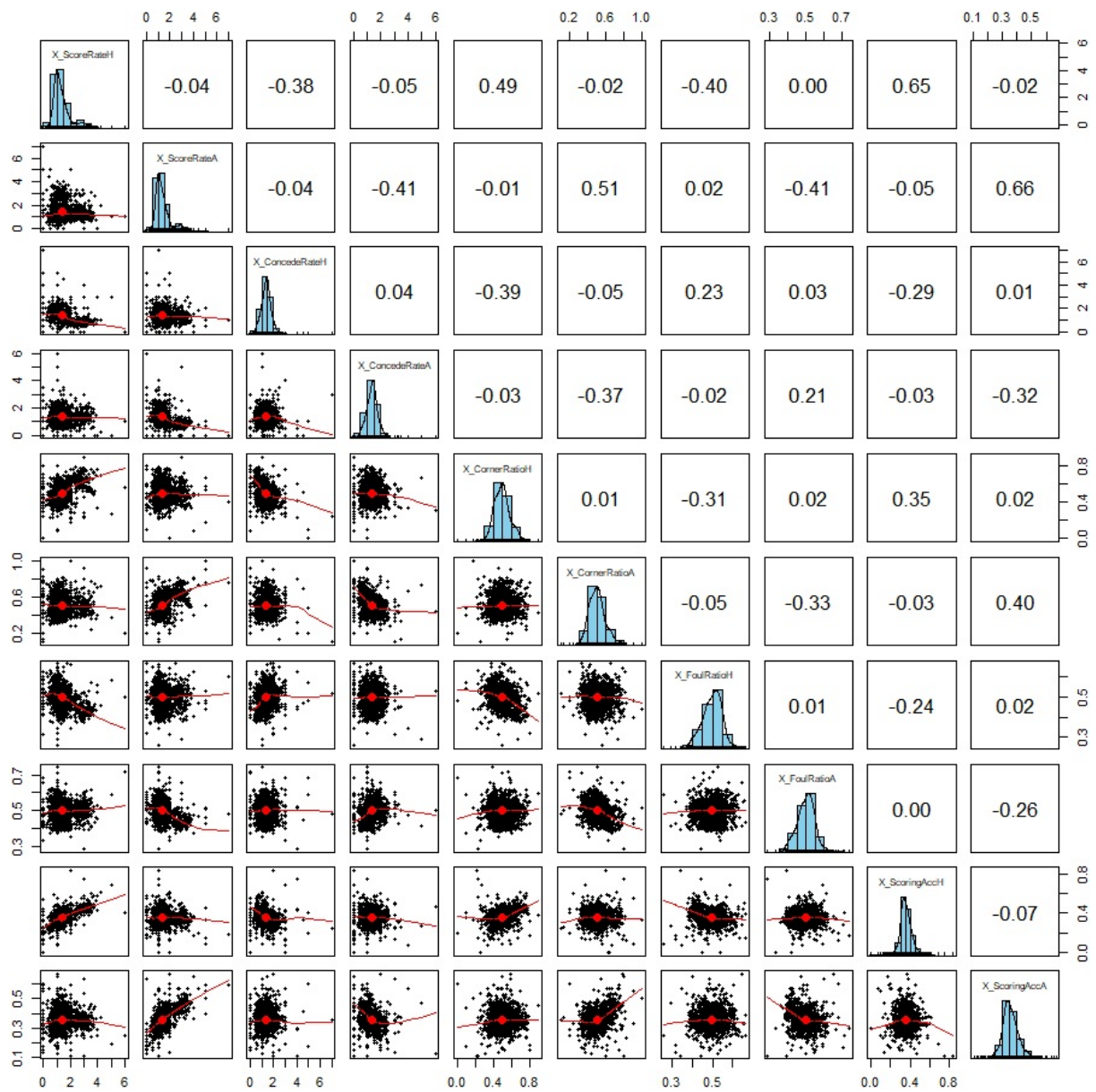
We will use this for predicting the amount of goals scored on a football match (`data/football.csv`). The dataset contains names of the home and away team, the date of the match, name of the league, home and away goals scored and parameters that describe properties of the two clashing teams. We will use the parameters that describe the play-style of the two teams to predict the number of goals scored by the home team. Complete and commented code for this example can be found in `code/poisson.R`, in this document we will touch only the most

interesting parts of this analysis.

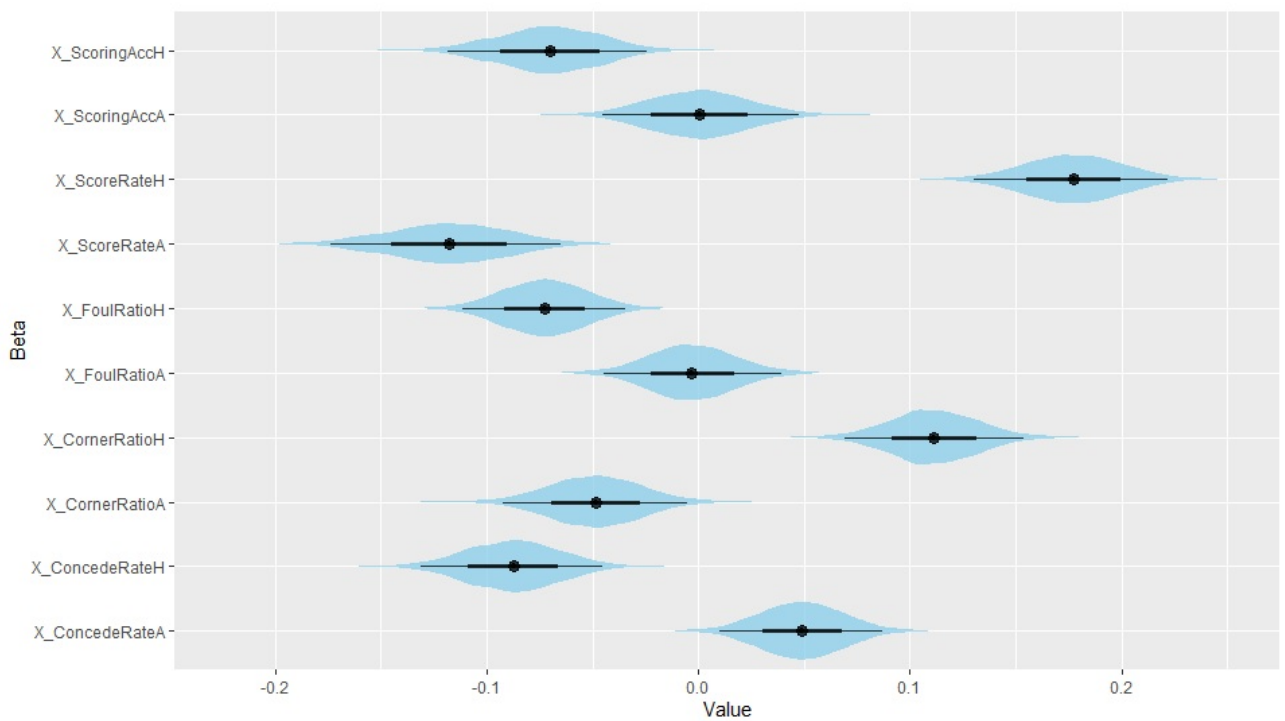
When preparing the input data we decided to standardize it. This often results in a better fitting experience and results that are easier to interpret. To do this, we used R's `scale` function. The left graph shows original independent variables while the ones on the right are standardized.



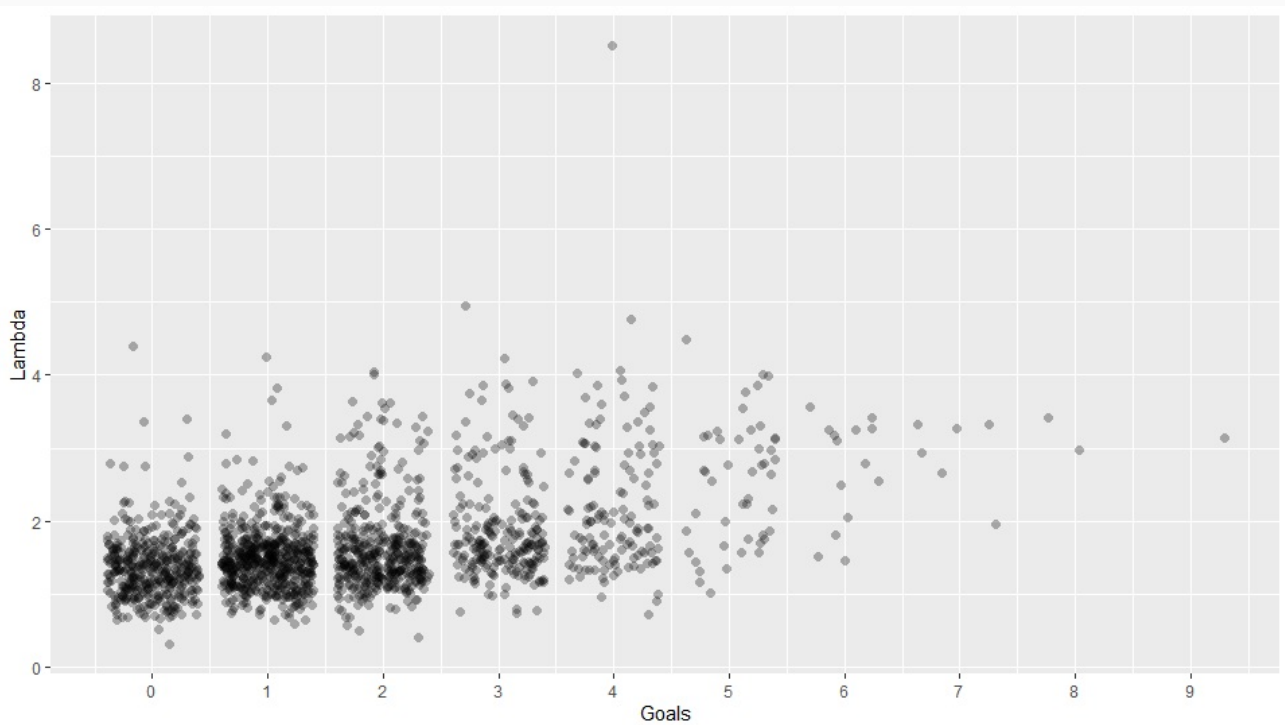
Once we standardize our independent variables we can check their distributions and correlations. The figure below shows no reason for concern so we can continue with our analysis.



There were no issues in the fitting process and basic diagnostics show no reasons for concern. A quick inspection of β coefficients (see the figure below) suggests that the results make sense – independent variables associated with the amount of goals scored by the away team (e.g., `X_ScoringRateA` and `X_CornerRatioA`) have less influence on predictions as the variables directly associated with the amount of goals scored by the home team (e.g., `X_ScoringRateH` and `X_CornerRatioH`). In a Bayesian setting we usually determine whether an independent variable has an influence (either positive or negative) on the dependent variable by checking whether the whole 95% HDI of its β coefficients posterior lies above or below 0.

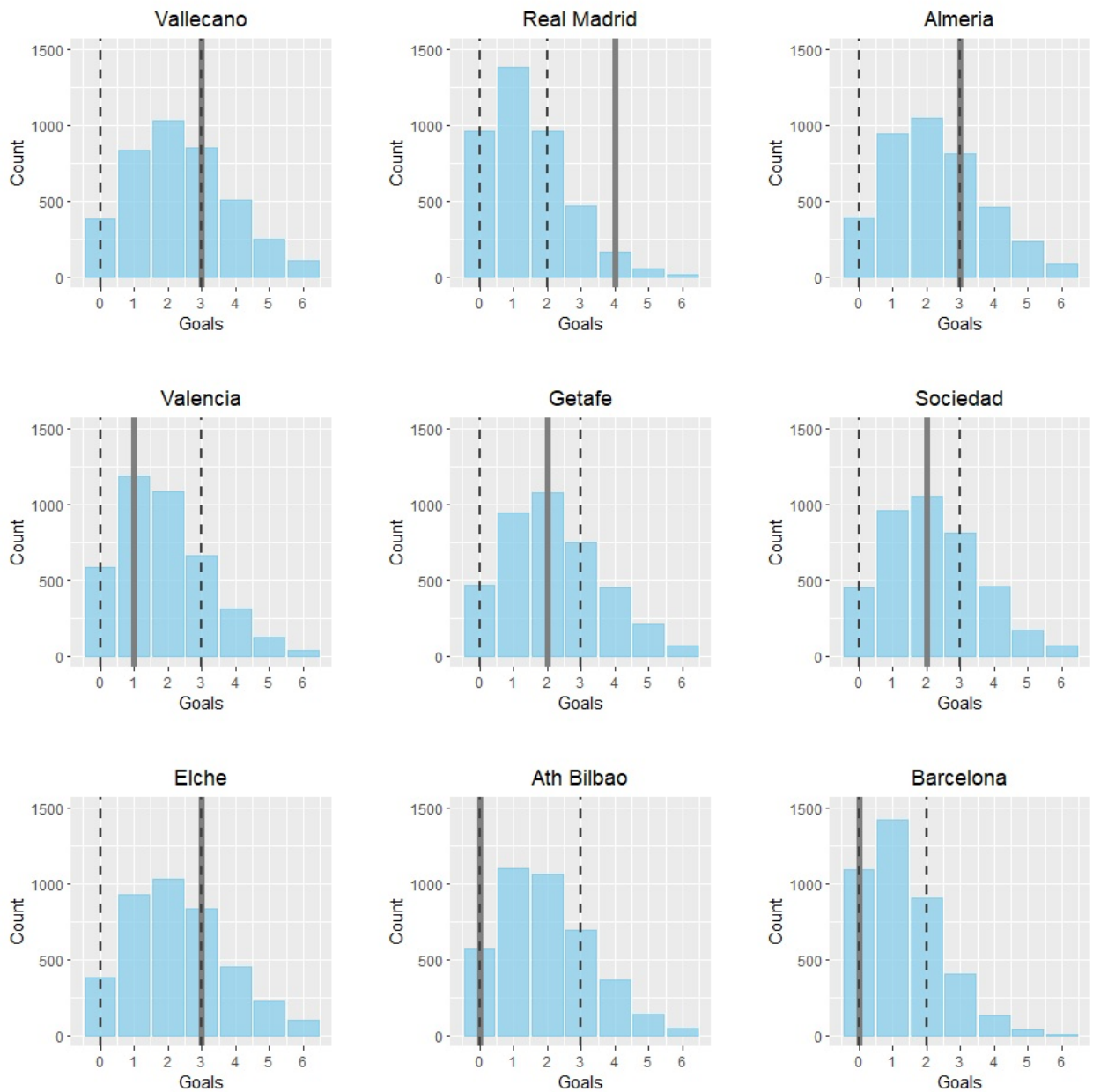


To inspect the quality of our model, we can compare how the estimated λ compares against the actual amount of goals scored by the home team. To make the visualization clearer we added some jitter to data points, all points above the number 0 represent matches where 0 goals were scored by the home team:



This figure suggests that there is a positive correlation between the λ parameter and the amount of goals scored by the home team. However, it seems like our model is a little conservative in its predictions – it will rarely predict games with a lot of scored goals. As also seen on the graph above such games are quite uncommon (in the majority of games the home team scored at most 2 goals).

To get a better understanding of our model let us compare a couple of predictions against the actual results of those games. To do this, we will compare the predictions and the actual amount of goals scored by the home team on the last 9 Atletico Madrid games that we have in our database.



The blue histogram in the background visualizes the posterior distribution of our predictions, the two dashed lines visualize the 25th and 75th percentiles of these posterior distribution and the thick solid line visualizes the actual result. We can see that in 8 out of 9 cases the actual amount of scored goals was inside this 50% HDI. Like expected, our model was way off the mark in the game where a lot of goals was scored.

// Categorical regression (multinomial logistic regression)

Categorical (or multinomial logistic) regression is the first choice model for nominal independent variables. It can be used for unordered and ordered nominal variables, although ordinal regression is more suitable in the latter case. The formulation of binary logistic regression as a log-linear model can be directly extended to categorical regression. In categorical regression the categorical distribution generalizes the Bernoulli and the softmax function generalizes the inverse logit:

$$y \mid X, \beta \sim \text{categorical}(\text{softmax}(X\beta')),$$

$$\text{softmax}(j, x_1, x_2, \dots, x_k) = \frac{e^{x_j}}{\sum_{i=1}^k e^{x_i}}, j \in [1, k].$$

Note here that β is not a vector (or a scalar) but a matrix. If we have k outcomes of our dependent variable y and n observations of m independent variables, then β will be a matrix with k rows and m

columns.

The classical categorical regression can have problems with identifiability (two or more parametrizations are observationally equivalent). Identifiability problems arise because softmax is invariant under adding a constant to each component of its input, thus the model is typically only identified if there is a suitable prior on the coefficients. To overcome this we can use a reference category, we implicitly did a similar thing in logistic regression. There we do not have β coefficients for both categories, β coefficients there are related only to one category ($y = 1$) while the other is set as a reference ($y = 0$). In Stan the categorical regression model thus looks like this:

```
data {
  int<lower=0> n;           // number of observations
  int<lower=0> m;           // number of independent variables
  vector[m] x[n];         // independent variables
  int<lower=0> k;           // number of outcomes
  array int<lower=1,upper=k> y; // dependent variables
}

transformed data {
  vector[m] zeros;
  zeros = rep_vector(0, m);
}

parameters {
  matrix[k-1, m] beta_raw; // k - 1, since one category is our reference
}

transformed parameters {
  matrix[k, m] beta;
  beta = append_row(beta_raw, zeros'); // last category is our reference
}

model {
  // fit
  for (i in 1:n)
    y[i] ~ categorical_logit(beta * x[i]);
    // or y[i] ~ categorical_softmax(beta * x[i]);
}
```

In this case every generated sample of the β parameter will be a matrix with m rows and k columns. Each value in that matrix will denote how a particular independent variable (out of m independent variables) relates to a particular outcome (out of k outcomes). Our code sample ([code/categorical.R](#)) analyses how a basketball's players role on the field and the distance from the rim influence player's shot selection. Some of the steps in data preparation might be somewhat new to you, so let us briefly explain our rationale behind them.

We start by loading our dataset, it has 3 columns – PlayerType, Distance, ShotType. Distance is a metric variable while player type (guard, forward or center) and shot type (above head, hook shot, layup) contain strings. Since the latter two are not strings of characters but factors (in our case both are nominal variables) we have to explicitly tell R that it should treat them as such.

```
# load data
data <- read.csv("../data/shot_types.csv", stringsAsFactors = TRUE)
```

Once our variable types are properly set, we can easily prepare matrices with independent and

dependent variables. The `model.matrix()` function can be used to create the so called model matrix (https://en.wikipedia.org/wiki/Design_matrix). In model matrices (or design matrices) each row represents an observation of our data, with the successive columns corresponding to the variables and their specific values for that object. To encode nominal variables model matrices use something called contrasts, you can think of contrasts as encoding of variables.

```
# library needed for n_distinct
library(dplyr)

# contrasts
contrasts(data$PlayerType) <- contr.treatment(n_distinct(data$PlayerType))

# display contrasts
contrasts(data$PlayerType)
```

```
##    2 3
## C 0 0
## F 1 0
## G 0 1
```

When constructing the model matrix we first have to specify which independent variables we want to encode, in the snippet above we use the `contrasts()` function to encode our PlayerType variable. Once we determine which variables have to be encoded, we can use the `model.matrix()` function to create the final model matrix. This matrix represents our independent variable related input data.

```
# note here that intercept is part of the model matrix (1st column always equals 1)
X <- model.matrix(~ Distance + PlayerType, data)

# show a couple of top rows to check if all is OK
head(X)
```

```
##   (Intercept) Distance PlayerType2 PlayerType3
## 1           1     3.99           1           0
## 2           1     0.86           1           0
## 3           1     7.25           0           1
## 4           1     4.47           1           0
## 5           1     7.25           0           1
## 6           1     4.95           1           0
```

One thing worth mentioning here is the first column (`(Intercept)`), as you see its value is always 1. In practice this means that we do not need an explicit intercept (α) in the formulation of our GLM, since the first β coefficient will be always multiplied by 1 and will thus effectively serve as the intercept. Therefore, intercept is often labeled as β_0 . To prepare the vector of dependent variables we can just use the ShotType column of our dataset. Since it is a factor it will be automatically translated into numerical variables before the fitting process.

```
# dependent variable
y <- data$ShotType
```



```
# show levels of y
levels(y)
```

```
## [1] "above head" "hook shot" "layup"
```

As you can see our dependent variable has three levels – above head, hook shot and layup – these will be encoded as 1, 2, 3, respectively. With this we finished the preparation of our input data, and we can start the fitting process. Since all diagnostics looked OK, we will again skip them. The next step is to extract our parameters.

```
# extract parameters
df_betas <- as_draws_df(fit$draws("beta"))
df_betas <- df_betas %>% select(-.chain, -.iteration, -.draw)

# beta matrix composed of sample means
betas <- matrix(colMeans(df_betas), nrow = 4, ncol = 3)
```

```
##           [,1]      [,2]      [,3]      [,4]
## [1,] -3.974213  6.325504 -1.408025 -5.589200
## [2,] -3.004120  5.516952 -1.552235 -5.939575
## [3,]  0.000000  0.000000  0.000000  0.000000
```

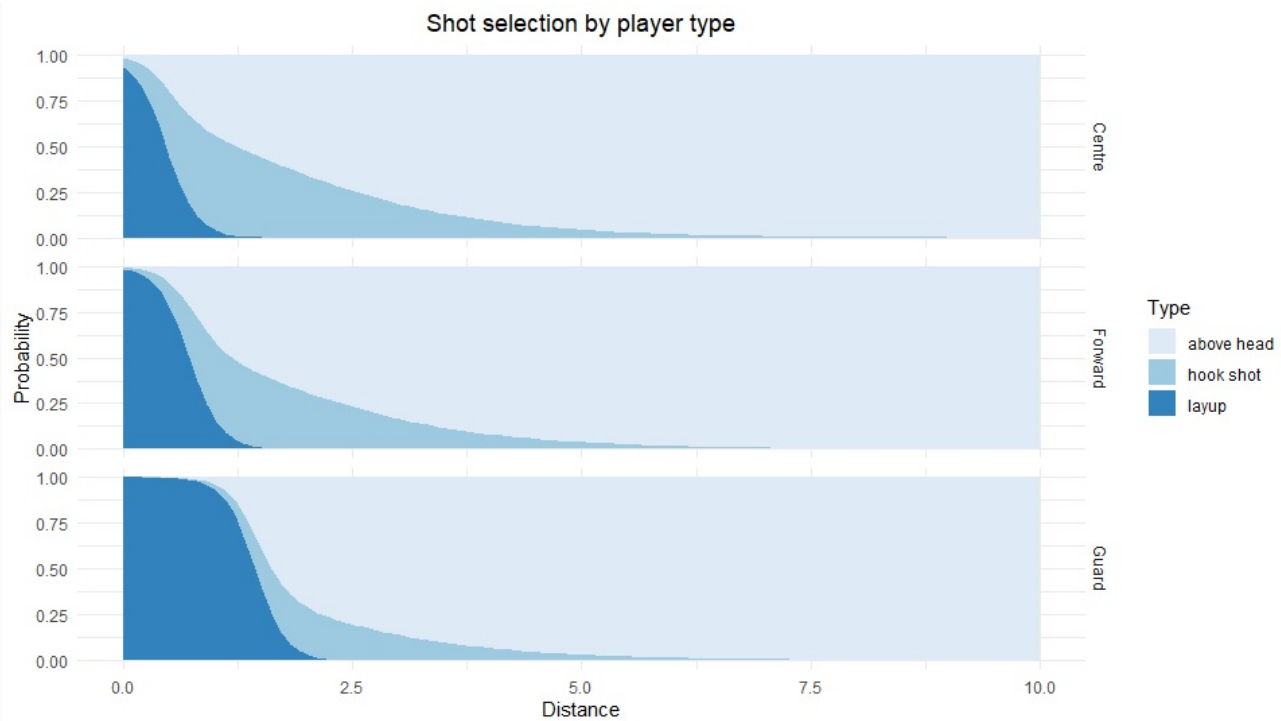
The matrix above represents mean β coefficient values. Four columns arise from our input variables ($m = 4$), while three rows arise from our outcomes ($k = 3$). So, the first column represents intercepts, the second column is associated with distance, the third with players playing at the forward position and the fourth with players playing at the guard position. The first row is associated with the above head shots, the second with hook shots and the third with layups. For example, the β value in the second row, and the third column denotes how probability of a hook shot changes if the player is a forward. Just like with other regressions, β coefficients are all we need for our analysis. The code snippet below calculates probabilities for each shot type when a center shoots from 0.5 m distance:

```
# x <- c(intercept, distance, forward, guard)
x <- c(1, 0.5, 0, 0)

# calculate probabilities
c_probs <- softmax(betas %*% x)
c_probs
```

```
## [1] 0.1995179 0.3513283 0.4491538
```

When a center is shooting from 0.5 m there is a 0.2 probability of an above head shot, 0.35 of a hook shot and 0.45 of a layup. The visualization below shows how probabilities of a shot type for each player type change with distance from the rim.



Note here that calculating the sample means for the beta matrix is not the true Bayesian way. The true Bayesian way would be to calculate a beta matrix for each of the samples and then work with that, this way we would have an estimate of uncertainty everywhere along the way. In this illustrative example (and some others that follow), we decided to work with means for brevity and clarity purposes.

An alternative, and arguably more elegant way of calculating shot type probabilities would be to use the `generated quantities` block.

// Gamma regression

Gamma regression is useful when our dependent variable resides on the positive side of real numbers. However, using the gamma distribution in a GLM is a bit trickier because it's two default parameters ν (shape) and λ (rate) do not directly represent the mean and the variance. The two parameters of the gamma distribution are traditionally called α and β , here we are using ν and λ to distinguish them from the intercept and β coefficients that will come into play later on:

$$y \mid \nu, \lambda \sim \text{gamma}(\nu, \lambda),$$

$$E[y] = \mu = \frac{\nu}{\lambda},$$

$$\text{Var}(y) = \phi = \frac{\nu}{\lambda^2}.$$

As you can see we cannot directly map the linear predictor to a gamma parameter. Fear not, we can fix this with some simple calculus. We can reparametrize the gamma distribution to fit our needs. There are two ways this is commonly done in the literature. The first:

$$\mu = \frac{\nu}{\lambda},$$

$$\nu = \mu\lambda.$$

Now we have an expected value parameter μ . Since gamma distribution resides on the positive real scale we can use a log link function. Putting everything together gets us:

$$y \mid \mu, \lambda \sim \text{gamma}(\mu\lambda, \lambda),$$

$$\log(\mu) = \beta_0 + \beta_1 x_1 + \dots + \beta_k x_k,$$

$$\mu = e^{\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k}.$$

And the second way is:

$$\mu = \frac{v}{\lambda},$$

$$\phi = \frac{v}{\lambda^2} = \frac{v}{\lambda} \frac{1}{\lambda} = \frac{\mu}{\lambda},$$

$$\lambda = \frac{\mu}{\phi},$$

$$v = \mu\lambda = \mu \frac{\mu}{\phi} = \frac{\mu^2}{\phi}.$$

Here, we again have an expected value parameter μ and also a variance parameter ϕ . We can again use a log link:

$$y \mid \mu, \phi \sim \text{gamma}\left(\frac{\mu^2}{\phi}, \frac{\mu}{\phi}\right),$$

$$\log(\mu) = \beta_0 + \beta_1 x_1 + \dots + \beta_k x_k,$$

$$\mu = e^{\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k}.$$

Based on the above, you should be able to develop a gamma regression model in Stan.

// Ordered logistic regression

Ordered logistic regression (also known as ordinal logistic regression or proportional odds model) is used for modeling variables on the ordinal scale. That is nominal variables for which there exists a meaningful ordering. The most common use case of ordered logistic regression is modeling preference (e.g., on a five-level Likert scale, ranging from strongly disagree to strongly agree).

Essentially, ordinal variable is just a categorization (multinomial prediction) problem with an additional constraint – responses have an exact ordering. Meaning that our model has to ensure a proper ordering of outcomes. The traditional solution to this problem is to use a cumulative link function. Suppose our dependent variable y has k outcomes and $j \in 1, \dots, k$, the cumulative probability of outcome j thus equals:

$$F_y(j) = \Pr(y = 1) + \Pr(y = 2) + \dots + \Pr(y = j) = \Pr(y \leq j).$$

With the logit regression we worked with log odds by using a logit link function to assure that probabilities are constrained to the $[0, 1]$ interval. We will use a similar approach in the ordered logistic regression, the main difference is that, since we are working with a cumulative link function, we also have to work with the log cumulative odds.

Suppose we have n observations of m independent variables stored in matrix X . Each row in X represents an observation vector and has an associated response stored in vector y . Log cumulative odds that value of y is lower or equal then the outcome j are thus:

$$\text{logit}(\Pr(y \leq j)) = \log\left(\frac{\Pr(y \leq j)}{1 - \Pr(y \leq j)}\right) = \alpha_j - X\beta,$$

$$\Pr(y \leq j) = \frac{1}{1 + e^{-(\alpha_j - X\beta)}},$$

where α_j is a cutpoint (threshold, “intercept”) unique to each possible outcome value j . Since outcomes are ordered $\alpha_1 < \alpha_2 < \dots < \alpha_{k-1}$ and the cumulative log odds of the largest variable are always infinity ($\log(1/(1 - 1))$) we do not need a parameter for it, which is why we need only $k - 1$ cutpoints. In the above equation $X\beta$ is a traditional linear model.

Notice that we are subtracting the linear term from each cutpoint, the reason being that when we decrease the log cumulative odds of every outcome value j below the maximum (k), this shifts probability mass upwards towards higher outcome variables, which is the desired outcome if β coefficients are positive. Finally, the probability that the outcome of y equals j is calculated as:

$$\Pr(y = j) = F_y(j) - F_y(j - 1) = \Pr(y \leq j) - \Pr(y \leq j - 1).$$

Fortunately, the implementation in Stan is easier. In the code below the vector of cutpoints `c` is declared as `ordered[k-1]`, which guarantees that `c[j]` is less than `c[j+1]`.

```
data {
  int n;                // number of observations
  int m;                // number of independent variables
  row_vector[m] x[n];   // independent variables
  int k;                // number of outcomes
  array[n] int<lower=1,upper=k> y; // dependent variables
}

parameters {
  vector[m] beta; // beta coefficients
  ordered[k-1] c; // cutpoints
}

model {
  for (i in 1:n)
    y[i] ~ ordered_logistic(x[i] * beta, c);
}
```

See `code/ordered.R` for a practical example of the ordered logistic regression. There we analyze how religion (yes/no), sex, college degree (yes/no) and country of residence (USA/Sweden) influence people’s beliefs whether their country does enough to fight poverty.

// And many more

There are many more GLMs out there that might come in handy when tackling a particular problem (e.g probit regression, ordered probit regression, binomial regression, inverse Gaussian regression, exponential regression...). Luckily the core concepts are the same in all of them, so equipped with the knowledge you gained today you should not have too many issues when implementing them. Some of the mentioned GLMs are already implemented in Stan, to take a look at all readily available regression models in Stan consult https://mc-stan.org/docs/2_25/stan-users-guide/regression-models.html.

An alternative to coding your own models is an R package called `rstanarm` that emulates other R model-fitting functions but uses Stan (via the `rstan` package) for the back-end estimation. The primary target audience is people who would be open to Bayesian inference if using Bayesian software were easier but would use frequentist software otherwise.

/ Recommended readings

- **Regression Models** in Stan User’s Guide (https://mc-stan.org/docs/2_25/stan-users-

<guide/regression-models.html>).

- **Chapter 6 (Overfitting, Regularization and Information Criteria) and 10 (Counting and Classification)** in McElreath R. (2016). Statistical Rethinking: A Bayesian Course with Examples in R and Stan. CRC Press.
- **Chapter 15 (Overview of the Generalized Linear Model)** in Kruschke J. K. (2015). Doing Bayesian Data Analysis: A Tutorial with R, JAGS, and Stan (2nd edition). Academic Press.

/ Additional readings

- van Erp S., Oberski D. L., & Mulder J. (2019). Shrinkage priors for Bayesian penalized regression, Journal of Mathematical Psychology, 89, 31–50.
- **Chapters 4, 7 and 9** in McElreath R. (2016). Statistical Rethinking: A Bayesian Course with Examples in R and Stan. CRC Press.
- **Part III** of Kruschke J. K. (2015). Doing Bayesian Data Analysis: A Tutorial with R, JAGS, and Stan (2nd edition). Academic Press.
- Wiki page about the generalized linear model (https://en.wikipedia.org/wiki/Generalized_linear_model).
- Wiki page about the linear regression (https://en.wikipedia.org/wiki/Linear_regression).

Processing math: 100%